

Team Control Number

For office use only

187

For office use only

F1 _____

F2 _____

F3 _____

F4 _____

Problem Chosen

B**2005 Mathematical Contest in Modeling (MCM) Summary Sheet**

(Attach a copy of this page to each copy of your solution paper.)

Type a summary of your results on this page. Do not include the name of your school, advisor, or team members on this page.

Summary

This paper is aimed to limit the traffic disruption by determining the optimal number of tollbooths per travel lane.

The two main causes of disruption are discussed. One is to wait for the vacancy of the tollbooth when it is occupied. The other is to wait to squeeze back down to one travel lane. Respectively, the whole system can be considered as a cascade of two queuing subsystems. And we define ‘optimal’ as the number of tollbooths per travel lane that makes the average waiting time minimal.

Based on the assumption that all the tollbooths are E-ZPass, we assess average service time of both queuing subsystems. We assume that the arriving time interval and the service time of the first queuing subsystem follow exponential distribution. Thus the first subsystem can be viewed as a M/M/N queuing system. Because the two queuing subsystems are cascaded, the arriving time of the second subsystem is just the departing time of the first. Then the second subsystem can be viewed as a G/M/1 queuing system.

For the second subsystem, the distribution of the arriving time cannot be expressed definitely. After analyzing it qualitatively, we use program simulation to get the answer. When analyzing the average waiting time of ‘one tollbooth per incoming travel lane’, we find it is more reasonable to assume that the service time follows Erlang distribution. Then using MATLAB program, we determine the optimal number of tollbooths per travel lane is 3 when the traffic flow is less than 1800 vehicles/h; while if the traffic flow becomes lager, congestion can’t be avoided no matter how many tollbooths per travel lane, thus the number of tollbooths doesn’t matter.

As for ‘one tollbooth per incoming travel lane’, it is better only when the toll service time is shorter than 2s.

Then, we generalize our model to a situation where cash-paying is also used, and conclude that 3 cash-paying tollbooths and 3 E-ZPass tollbooths per travel lane are optimal if 70% vehicles use E-ZPass tollbooth and 30% use cash-paying tollbooth.

At last, based on our model, some recommendations are offered.

Pass the Plaza more Quickly

电子科技大学

叶磊2201311004余侠2206202021张婧2202401011

Introduction

On heavily-traveled toll roads such as the Garden State Parkway, Interstate 95, and so forth, toll plazas disrupt traffic at intervals. In order to limit the amount of disruption, a much larger number of tollbooths are usually provided in the toll plaza than the number of travel lanes entering it. To go through a toll plaza, vehicles have to slow down, queue up for payment, pay the toll, wait to squeeze back down to travel lanes...it is so annoying and time-wasting that a pleasant trip is spoiled!!! In order to minimize the amount of this kind of traffic disruption, we should determine the optimal number of tollbooths per incoming travel lane.

We think that, for motorists, the most annoying disruption is ‘waiting’, rather than the payment itself. This is because as long as the toll plaza exists, the payment exists, while ‘waiting’ is a kind of time-waste. Thus the primary cause of motorist annoyance is ‘waiting’.

Then we investigate the ‘waiting’. It exists primarily in two situations:

1. Waiting for the vacant tollbooth. When the tollbooth is occupied, motorists have to drive more slowly or even stop to wait for the finish of the current payment.
2. Waiting for squeezing back down to the travel lane. Vehicles from several tollbooths are required to squeeze back down to one travel lane. Some vehicles have to decelerate or even stop to let others go first.

So our goal is to find the optimal number of tollbooths per travel lane, which can minimize the motorist ‘waiting’.

Terms and Symbols

ideal time : the time spent in the toll plaza in an ideal situation (no waiting).(s)

additional delay t_d : the time additionally wasted in a general case comparing with the **ideal time**, that is, the time wasted in waiting.(s)

average additional delay t_{da} : the average value of **additional delay**.(s)

toll waiting time t_{w1} : the time to wait for the vacancy of the tollbooth when it is occupied. (s)

crossing : the location where the flow of vehicles is required to squeeze back down to a travel lane.

crossing waiting time t_{w2} : the time to wait for the previous vehicle to pass the **crossing**.(s)

plaza-arriving time: the time when a vehicle arrives at the toll plaza. That is the arriving time for the first subsystem.(s)

crossing-arriving time : the time when a vehicle arrives at the **crossing**. It is the arriving time for the second subsystem; in the meantime, it is the time when a vehicle finishes its toll collection service. (s)

tollbooth-departing time: the time when a vehicle leaves the tollbooth. That is the departing time for the first subsystem.(s)

crossing-departing time: the time when a vehicle leaves the **crossing**. That is the departing time for the second subsystem.(s)

plaza-arriving time interval: the time interval between two immediate vehicles arriving at the toll plaza. (s)

crossing-arriving time interval: the time interval between two immediate vehicles arriving at the **crossing**.(s)

traffic flow Q : the number of vehicles arriving at the toll plaza per hour per travel lane.(vehicles/h)

toll service time: when the tollbooth is busy, the time interval between two vehicles accepting the toll service. That is the service time of the first subsystem. (s)

crossing service time: when the **crossing** is busy, the time interval between two vehicles accepting the passing-by service. That is the service time of the second subsystem. (s)

Assumptions:

1. All tollbooths are E-ZPass. That is, vehicles can pay the toll by just passing by the tollbooths at a relatively slower speed.
2. The **arriving time interval** between two immediate vehicles follows exponential distribution.
3. The **toll service time** t_{s1} follows exponential distribution.
4. The **crossing service time** t_{s2} follows exponential distribution. And its expectation is 2 s.
5. Different types of vehicles are viewed as the same, that is, we take no account of the differences due to different types of vehicles. For example, we consider that they have the same average length, velocity, etc.
6. The average length of different vehicles is 5 m.
7. The average velocity when vehicles are accepting toll collection service (passing by the tollbooth) is 7 mph, that is 11.2 km/h.
8. The minimal distance from the vehicle accepting service to the one waiting for service at the tollbooth is 5 m.
9. For each travel lane, the number of tollbooths is the same. Vehicles from one travel lane can only use its own tollbooths, but can't choose tollbooths deployed for other lanes. (Otherwise, there may occur an accident.)
10. There is no traffic accident.
11. There is no difference between different travel lanes.

Queuing Model

Analysis

Just imagine an 'ideal' situation. There are no other vehicles in the toll plaza ahead of yours, so you can drive freely: slows down when approaching, pays the toll, then speeds up and departs. We define the time spent in an 'ideal' situation as the **ideal time**. And we assume the **ideal time** is the same for a vehicle no matter which tollbooth it chooses.

But in a general case, you can't be so lucky, that is, there is always someone ahead of you and preventing you from driving freely. For example, the tollbooth is now occupied by a car, so you have to

make your car slower than the ‘ideal’ situation or even stop and wait until the previous car’s departure. We define that, the time additionally wasted in a general case comparing with the **ideal time**, to be the **additional delay** (denoted as t_d). So the total time spent in a toll plaza is the sum of the **ideal time** and the **additional delay** (if there is any).

As for the two components of the total time, the **ideal time** and the **additional delay**, we can only change the latter. Thus in order to limit the amount of traffic disruption caused by the toll plaza, we can only shorten the **additional delay** by determining the optimal number of tollbooths to deploy in a toll plaza. Thus, we define the ‘**optimal**’ to be the minimal **additional delay**. (Note that we can’t decrease the ideal time by deploying more tollbooth.)

Then we analyze the **additional delay**. First we define the location where the flow of vehicles is required to squeeze back down to a travel lane as the **crossing**. The **additional delay** can be divided into two parts: one is to wait for the vacancy of the tollbooth when it is occupied, and we define it as **toll waiting time** (denoted as t_{w1}); the other is to wait for the previous vehicle to pass the **crossing**, and we define it as **crossing waiting time** (denoted as t_{w2}). (Note that the ‘wait’ is not necessarily ‘stop’. If the tollbooth or the **crossing** is occupied, the motorist will drive more slowly because it can’t accept service as soon as it arrives: even if it arrives earlier, it still has to stop to wait. So the ‘wait’ also includes the time wasted in the ‘more slowly’ driving.) Thus we get:

$$t_d = t_{w1} + t_{w2} \quad (1)$$

And our goal is to minimize the average t_d , denoted as t_{da} .

Now, we focus on analyzing the causes of t_{w1} and t_{w2} respectively.

1. The cause of **toll waiting time** t_{w1} . t_{w1} is due to the occupation of the tollbooth when the next vehicle arrives. This occupation can be viewed as the acceptance of a kind of service: the toll collection service. We define the time a vehicle spend to accept this service as **toll service time** (denoted as t_{s1}).
2. The cause of **crossing waiting time** t_{w2} . t_{w2} is due to the occupation of the **crossing** when the flow of vehicles squeezes back down to the travel lane. This occupation can be viewed as the acceptance of a kind of service: the passing-by service. We define the time a vehicle spend to accept this service as **crossing service time** (denoted as t_{s2}).

So far, we have viewed the whole process a vehicle comes through as the acceptance of two services: the toll collection service and the passing-by service. We also define respectively the waiting time and the service time for the two services. Thus we divide the whole process a vehicle comes through into two subsystems, each of which is a queuing system, and these two subsystems are cascaded (Figure 1). We consider that as soon as vehicles leave the first subsystem they enter the second.

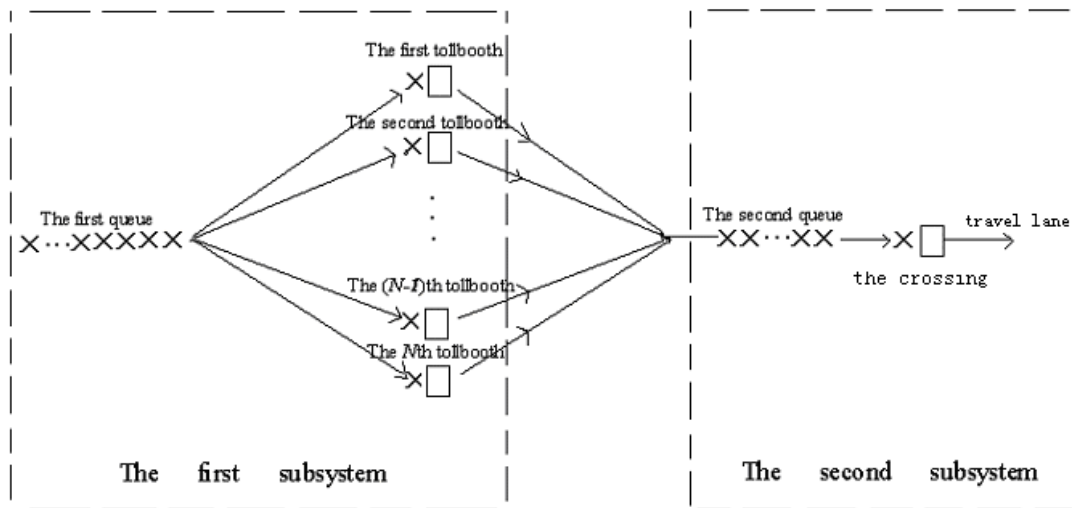


Figure 1. The
two subsystems

Service Time

In this subsection, we calculate the expectations of the two *service times*.

Toll service time t_{s1} is the time a vehicle occupies the tollbooth to accept the toll collection service.

We assume that all tollbooths are E-ZPass, that is, vehicles can pay the toll by just passing by the tollbooths at a relatively slower speed. Because of the Highway Code about the ‘safe distance’, a vehicle can’t accept the toll collection service as soon as the previous vehicle completes its service. That is, the vehicle behind cannot accept ‘service’ until the previous one has driven to the point a ‘safe distance’ away from the tollbooth. We assume that the safe distance is 5 m, and the average length of vehicles is 5 m, thus the tollbooth can serve the next vehicle only when the previous vehicle has driven 10 m (the sum of the safe distance and the average length of vehicles), and before this the tollbooth can’t serve another vehicle. So we determine t_{s1} as the time a vehicle covers the distance 10 m.

Because 10 m is quite a short distance, we reckon the vehicle’s passing speed as a constant, and we assume it is 7mile/h. So the expectation of t_{s1} is:

$$E[t_{s1}] = \frac{\text{the safe distance} + \text{the average length}}{\text{the passing speed}} = \frac{10\text{m}}{7\text{mile/h}} = 3.214\text{s}$$

As for **crossing service time** t_{s2} , it is the time a vehicle occupies the **crossing** and accepts the passing-by service. Considering safety, if two vehicles pass by the **crossing** very close to each other, it is very likely to trigger an accident. So in order to guarantee safety, a vehicle can pass by the **crossing** only when the previous vehicle has driven some time earlier. We suppose that a vehicle can pass by a location at least 2s after the previous one passes by the **crossing**, that is, a second vehicle can accept the

passing-by service at least 2s after the previous one accepts. We consider this time to be the **crossing service time**, its expectation is 2s,

$$E[t_{s2}] = 2s$$

The First Subsystem: Toll Subsystem

In this subsection, we investigate the first subsystem using the queuing theory. We assume **arriving time interval** and **toll service time** follow exponential distribution. Thus, the first subsystem is an $M/M/N$ queuing system. ($M/M/N$ is a system where both arriving time interval and service time follow exponential distribution and the number of service counters is N .) The average waiting time of this kind of queuing system can be calculated by [Ref 1]

$$E[t_{w1}] = \frac{\rho_1^N p_0 E[t_{s1}]}{N \cdot N! (1 - \rho)^2} \quad (2)$$

Where

$$\begin{cases} \rho_1 = QE[t_{s1}] \\ \rho = \frac{QE[t_{s1}]}{N} \\ p_0 = \left(\sum_{k=1}^{N-1} \frac{\rho_1^k}{k!} + \frac{\rho_1^N}{N!} \cdot \frac{1}{1 - \rho} \right)^{-1} \end{cases}$$

Q is the average **traffic flow** entering the toll plaza per travel lane. $E[t_{s1}]$ is the expectation of **toll service time**. N is the number of tollbooths per travel lane.

We assume that $Q = 1100 \text{ vehicles/h}$, and we have obtained that $t_{s1} = 3.2 \text{ s}$. Set $N = 1, 2, K, 10$, and we can obtain 10 $E[t_{w1}]$ corresponding to each N respectively (Table 1)

Table 1. $E[t_{w1}]$ for $N = 1, 2, K, 10$

number of tollbooths	1	2	3	4	5	6	7	8	9	10
average waiting time	140.8	1.00	0.14	0.02	0.003	0.00	0.00	0.00	0.00	0.00

The Second Subsystem: Crossing Subsystem

We assume the **crossing service time** follows exponential distribution. Thus the second subsystem can be viewed as a $G/M/1$ queuing system. ($G/M/1$ is the system where arriving time interval follows a general distribution while service time follows Exponential distribution and the number of service counters is 1). So the average waiting time of this kind of queuing system is: [Ref 2]

$$E[t_{w2}] = \frac{\sigma}{E[t_{s2}](1 - \sigma)} \quad (3)$$

Where $E[t_{s2}] = 2s$ and σ can be calculated by solving the following equation:

$$\sigma = \int_0^{\infty} e^{-(E[t_{s2}] - E[t_{s2}]\sigma)t} g(t) dt$$

Where $g(t)$ is the probability density function of the ***crossing-arriving time interval***. Note that $g(t)$ is also the probability density function of the interval of ***tollbooth-departing time*** (we define the time when a vehicle leaves the tollbooth as ***tollbooth-departing time***.)

Combining (1)(2)(3), our goal is:

$$\begin{aligned} \min \quad & t_{da} = E[t_{w1}] + E[t_{w2}] \\ \text{s.t.} \quad & \begin{cases} E[t_{w1}] = \frac{\rho_1^N p_0 E[t_{s1}]}{N \cdot N! (1-\rho)^2} \\ \rho_1 = QE[t_{s1}] \\ \rho = \frac{QE[t_{s1}]}{N} \\ p_0 = \left(\sum_{k=1}^{N-1} \frac{\rho_1^k}{k!} + \frac{\rho_1^N}{N!} \cdot \frac{1}{1-\rho} \right)^{-1} \\ E[t_{w2}] = \frac{\sigma}{E[t_{s2}](1-\sigma)} \\ \sigma = \int_0^{\infty} e^{-(E[t_{s2}] - E[t_{s2}]\sigma)t} g(t) dt \end{cases} \end{aligned} \quad (4)$$

Note that N is the only variable and $g(t)$ can be determined by N .

The Effect of N on t_{da}

We have derived the goal expression (4). But it is a pity that we can't derive $g(t)$ because the distribution of the ***crossing-arriving time interval*** is too complex. So we use Monte-Carlo method to sketch the distribution function of the ***crossing-arriving time interval*** (denoted as $G(t)$). (Figure 2)

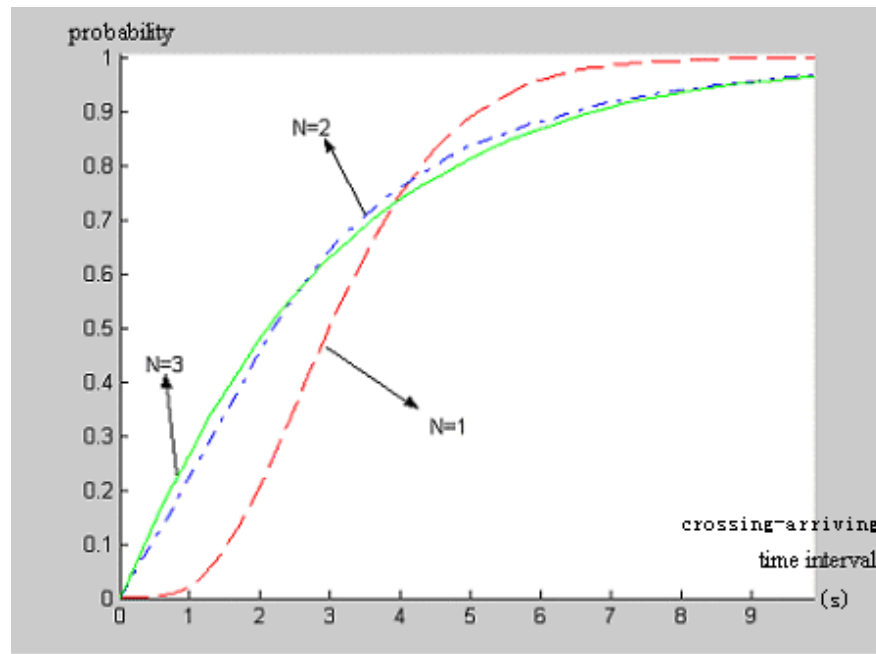


Figure 2. $G(t)$ for $N=1,2,3$
(N is the number of tollbooths per travel lane)

From Figure 2, we can approximately sketch the probability density function $g(t)$ (Figure 3).

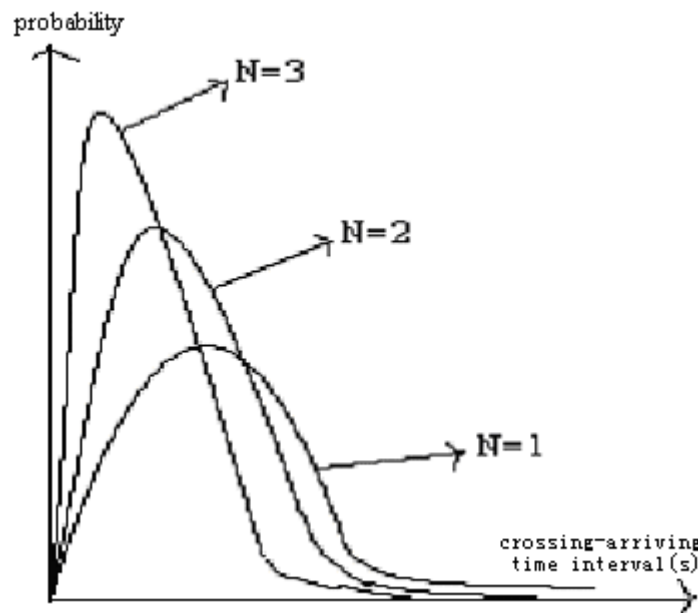


Figure 3. The probability density function $g(t)$ for $N=1,2,3$
(N is the number of tollbooths per travel lane)

From Figure 3 we can see that with N increasing, the peak of the curve is closer to the probability-axis, that is, for the **crossing**, the probability of immediate arriving is greater, thus the probability of waiting for the **crossing** is greater. So we conjecture that the greater N is, the longer $E[t_{w2}]$ is. And from the results we have derived (Table 1), $E[t_{w1}]$ becomes shorter with N increasing. Thus, we conjecture there exists a minimal value of t_{da} .

We sketch the distribution function when $N=3,4,5$ (Figure 4). From this figure, we can see the three

curves almost coincide with each other, which leads to little change in $E[t_{w2}]$. This can be explained by the fact that in most cases, cars use only the 3 of the tollbooths, and the others are usually vacant.

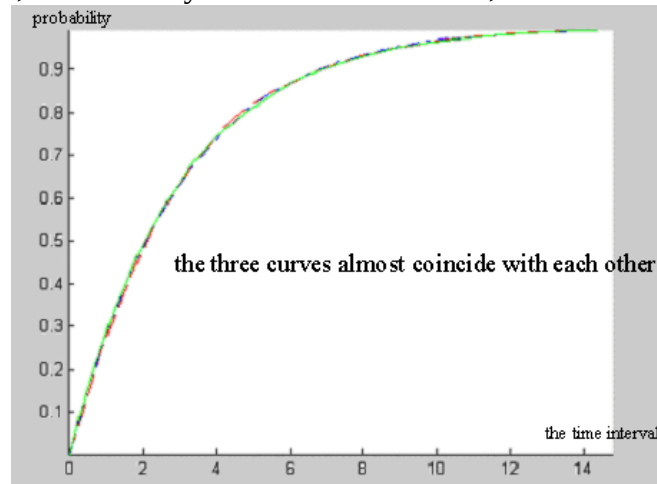


Figure 4. $G(t)$ for $N=3,4,5$

(N is the number of tollbooths per travel lane)

Besides, from Table 1, we know that t_{w1} changes little with N increasing when $N \geq 3$.

Consequently, we can conclude that when N is greater than 3, increasing N can have little effect on the two waiting time t_{w1} and t_{w2} , i.e., when $N \geq 3$, t_{da} is almost invariable.

Based on the analysis in this subsection, it can be concluded that t_{da} may have a minimal value and it will change little with N increasing when $N \geq 3$.

Simulation model

Consider the toll plaza as a system. This system consists of two subsystems. These two subsystems are two queuing systems, and they are FCFS (First Come First Served). The first queuing system is the tollbooths, and the second queuing system is the **crossing**. When a vehicle arrives at the toll plaza, it will wait for a vacant tollbooth, accept toll collection service, wait for the crossing, accept passing-by service, and leave the crossing. Since we have known that the **plaza-arriving time interval** (we define the time interval between two immediate vehicles arriving at the toll plaza as **plaza-arriving time interval**), the **toll service time** and the **crossing service time** follow exponential type distribution. So, we can get the **plaza-arriving time** (we define the time when a vehicle arrives at the toll plaza as the **plaza-arriving time**.) and the **tollbooth-departing time** by computer simulation using queuing theory. But one vehicle, which arrives at the toll plaza ahead of another, may leave the tollbooth behind it. This is because that there are several tollbooths and the toll service time is stochastic and is different for different vehicles. That is, the **tollbooth-departing time** may not be ordered as the order of the **plaza-arriving time**. Since we consider the **tollbooth-departing time** as the **crossing-arriving time** (we define the time when a vehicle finishes its toll collection service as the **crossing-arriving time**.) for the second queuing system (the **crossing**), and the **crossing** is FCFS. So, we have to sort the **tollbooth-departing time** (smallest first). We can use queuing theory again!!! So, we can obtain the **crossing-departing time** (we define the time when a vehicle leaves the **crossing** as **crossing-departing time**.) by computer simulation. Using the following formula, we can get the **additional delay** (t_d):

$$t_d = \text{crossing departing time} - \text{plaza arriving time} - t_{s1} - t_{s2}$$

Where t_{s1} is **toll service time**, and t_{s2} is **crossing service time**.

Based on the analysis above, we design our simulation arithmetic as follows (in Pascal):

1. Produce vehicles **arriving time interval**, **toll service time** and **crossing service time** for n vehicles, based on exponential type distribution Get **plaza-arriving time** according to vehicles **arriving time interval**.
2. **for** $i = 1$ **to** n ,
 - find the tollbooth which finishes the previous toll collection service earliest and record the time t when the service work is finished. Let the i -th vehicle go through this tollbooth.
 - if** the **plaza-arriving time** for the i -th vehicle $\geq t$.
 - tollbooth-departing time** for the i -th vehicle = **plaza-arriving time** for the i -th vehicle + **toll service time** for the i -th vehicle
 - else**
 - tollbooth-departing time** for the i -th vehicle = t + **toll service time** for the i -th vehicle
 - end**
3. Sort the **tollbooth-departing time** (smallest first) (bubble sort). And make it the **crossing-arriving time**.
4. **for** $i = 1$ **to** n
 - find the **crossing-departing time** for the $(i - 1)$ -th vehicle T
 - if** the **crossing-arriving time** for the i -th vehicle $\geq T$
 - crossing-departing time** for the i -th vehicle = **crossing-arriving time** for the i -th vehicle + **crossing service time** for the i -th vehicle
 - else**
 - crossing-departing time** for the i -th vehicle = T + **crossing service time** for the i -th vehicle
 - end**
5. Find the **additional delay** for each vehicle
 (**additional delay** for the i -th vehicle = **crossing-departing time** for the i -th vehicle – **plaza-arriving time** for the i -th vehicle – **toll service time** for the i -th vehicle – **crossing service time** for the i -th vehicle)
6. Get the average **toll waiting time** and the average **additional delay** and return.

Using the arithmetic above, we program in MATLAB. Deploy one tollbooth per travel lane, the simulation result (Appendix 1) shows that the second queuing system contributes greatly to the average **addition delay**. But considering the fact, because there is only one tollbooth per travel lane, there is no need for 'squeezing back'! So the **crossing service time** is very short and is usually shorter than the **toll service time**, that is, the first queuing system contributes greatly to the **additional delay**. Anatomizing this apparent difference between our simulation results and the fact, we find the cause is that we suppose the **toll service time** follows exponential type distribution. For this kind of distribution, the probability near zero is very great; but the fact is that the **toll service time** fluctuates near its expectation (nonzero), that is, the probability near the expectation is very great while the probability near zero is little. Knowing the characteristic of **toll service time**, we conclude that it is more reasonable to assume that the **toll service time** follows Erlang distribution. For the same reason, we assume that the **crossing service time** also follows Erlang distribution. So, in the remaining part of our paper, we all assume that the **toll service time** and the **crossing service time** follow Erlang distribution.

Now, our focus is to get the optimal number of tollbooths per travel lane. Using our modified MATLAB program, we can get the average **additional delay** according to input parameters: **traffic flow** and the number of tollbooths per travel lane. Set the **traffic flow** a reasonable value, and increase the

number of tollbooths gradually from 1. Compare the average **additional delay** for different number of tollbooths, we can find the optimal.

Given the fluctuation of program simulation, we modify the ‘optimal’ in program as:

If building the (n+1)-th tollbooth leads to a decrease by lower than 5% or even an increase in the average **additional delay**, we shouldn’t build it, and we deploy n tollbooths per travel lane.

Results & Analyses

In our modified simulation model, we program in MATLAB. Let the **traffic flow** be 1100 vehicles/h. The average **additional delay** for different number of tollbooths per travel lane is shown in Table 1.

Table 1. Average **additional delay** for different number of tollbooths

Number of Tollbooths	1	2	3	4	5	6	7	8	9	10	...
Average additional delay	164.12	2.21	2.0	1.94	1.85	1.76	1.88	1.81	1.74	1.89	...

From Table 1 and our definition about ‘optimal’ we find that the optimal number of tollbooths per travel lane is 3 when traffic flow is 1100 vehicles/h.

For other six **traffic flows** (less than 1800 vehicles/h), the optimal number of tollbooths per travel lane is shown in Table 2 (The average **additional delay** is in Appendix 2):

Table 2. The optimal number of tollbooths for different **traffic flows**

traffic flow	400	600	800	1000	1200	1400
optimal number	3	3	3	3	3	3

From Table 2, we can see that the optimal number for these 6 different **traffic flows** is always 3. So, we say that when the **traffic flow** is less than 1800 vehicles/h, the optimal number of tollbooths per travel lane is 3. But when the **traffic flow** reaches 1800 vehicles/h (1vehicle/2s), the **crossing** is always busy because the expectation of the **crossing service time** is just 2s. Thus the crossing becomes a bottleneck. Under this condition (or greater **traffic flow**), the optimal number determined by our program is no longer 3. But no matter how many tollbooths per travel lane, the average **additional delay** is too long (thousands of seconds) to bear. And if the **traffic flow** is constantly 1800 vehicles/h (or greater), the average **additional delay** will become infinite!!! It is because if there is only one tollbooth per travel lane, the tollbooths becomes the primary bottleneck; if there is more than or equal to two tollbooths per travel lane, the **crossing** becomes a bottleneck. That is, no matter how many tollbooths per travel lane, there will be a block. In this situation, the number of tollbooths per travel lane doesn’t matter.

One Tollbooth per Incoming Travel Lane

This part is to find out under what conditions one tollbooth per incoming lane will be better than more tollbooths per lane.

From the analysis above, we know that the traffic flow has little effect on the optimal number of tollbooths. So the primary factor is the **toll service time**.

To find out how **toll service time** affects the optimal number, we first analyze the system using the same method as in ‘the Effect of N on t_{da} ’. We find that the distribution of G(t) is almost the same as that of **plaza-arriving time interval** (Appendix 3) and the minimal t_{da} can be assessed by $E(t_{w2})$.

Let:

$$E(t_{w1}) < E(t_{w2})$$

Because the distribution of $G(t)$ is almost the same as that of *plaza-arriving time interval*, we can obtain:

$$E[t_{s1}] < E[t_{s2}]$$

So we get the condition is

$$E[t_{s1}] < 2$$

More General Case

In our previous model, we assume that all the tollbooths are E-Z pass. But in most toll plazas in America several types of toll collection are offered for motorists to choose. Now, we take into account other toll-collection types. We assume that 70% of the vehicles will choose E-Z pass, while 30% prefer to pay cash. We also assume that the expectation of *toll service time* for cash payment is 8 s [Ref 3]. In this situation, we just need to amend our arithmetic in ‘Simulation Model’:

1. Produce a random number that follows uniform distribution between 0 and 1. If it is smaller than 0.7, we think the vehicle will choose E-Z pass. Otherwise it will pay cash.
2. In step 3 of our arithmetic, we just need to find the tollbooth that finishes service work of the same type earliest.

The other parts are the same as before.

And the *traffic flow* is also 1100 vehicles/h. Using MATLAB program, we get the optimal number is 6: 3 E-Z pass, 3 cash-paying, and the average *waiting time* is 2.1 s.

Strengths and Weakness

Our model can be used on other kinds of tollbooths, for example, cash-paying tollbooth, cried card tollbooth, etc. For these tollbooths, we just need to modify the *toll service time*.

The optimal number of tollbooths per travel lane will not change with *traffic flow* if the type of toll service does not change. *Traffic flow* only has effect on the *average waiting time*.

The main weakness is that we don’t consider the interaction of different incoming travel lanes. That is to say the car can’t change their lanes.

Recommendations

1. There is no necessary to set too many tollbooths for each travel lane.
2. The best way to cut down the *additional delay* is to find a quicker tollbooth and use one tollbooth per lane.

References:

- 1.Lu Chuanlai.1994.Queuing Theory. Beijing University of Posts and Telecommunications:94.
- 2.Lu Chuanlai.1994.Queuing Theory. Beijing University of Posts and Telecommunications:185.
- 3.Study in Tollbooths Deploying and Safety. www.ti.js.cn/jsjtkj/j2000205/04-06.htm.

Appendix 1

Average waiting time for one tollbooth per travel lane when the service time follows Exponential

distribution.

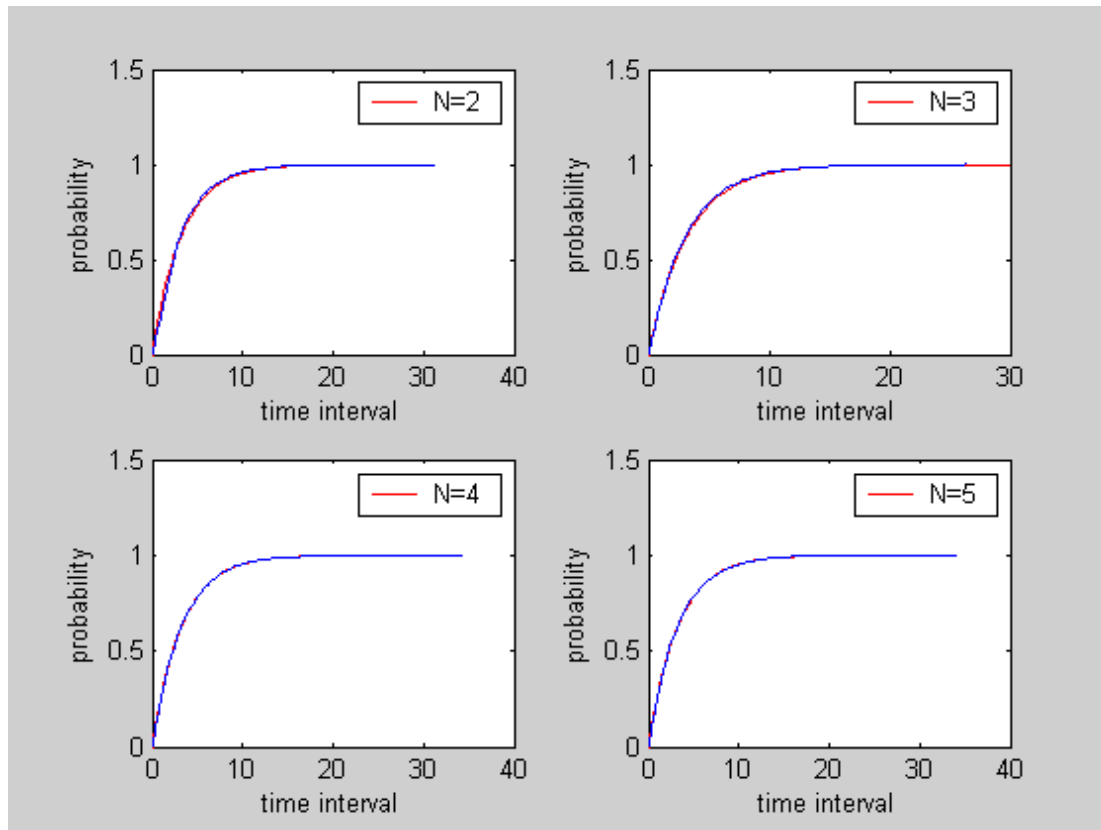
traffic flow	100	200	300	400	500	600	700	800	900	1000
toll waiting time	0.30	0.66	1.15	1.80	2.81	3.66	5.36	7.24	11.44	19.00
waiting time	0.43	0.92	1.59	2.36	3.57	4.69	6.62	8.90	13.48	21.34

Appendix 2

Average Waiting Time for Different Traffic Flow

	1	2	3	4	5	6	7	8	9	10
400	1.1564	0.3732	0.3320	0.3236	0.3528	0.3580	0.3372	0.3551	0.3360	0.3461
600	2.4525	0.6821	0.6293	0.6027	0.5839	0.6232	0.6146	0.6001	0.5879	0.6250
800	5.2878	1.0541	0.9925	1.0100	1.0246	1.0054	0.9726	0.9307	1.0216	0.9051
1000	14.7624	1.6893	1.5410	1.5419	1.6397	1.4041	1.6111	1.6325	1.4697	1.5794
1200	1087.4	2.8	2.3	2.5	2.3	2.3	2.4	2.3	2.5	2.4
1400	3073.2	4.4	4.1	4.3	3.9	3.9	4.1	3.8	4.2	4.4

Appendix 3: the Monte-Carlo simulation. The red is the exponential distribution. the blue is the simulation cure.



Appendix 4: Erlang function

% This function is to produce 5-Erlang distribution random number

function y=Errand(lamada,n);

% this function is to return n 5-Erlang randoms

r=rand(1,n);

y1=(-1/lamada/5)*log(r);

```

r=rand(1,n);
y2=(-1/lamada/5)*log(r);
r=rand(1,n);
y3=(-1/lamada/5)*log(r);
r=rand(1,n);
y4=(-1/lamada/5)*log(r);
r=rand(1,n);
y5=(-1/lamada/5)*log(r);
y=y1+y2+y3+y4+y5;

```

Appendix 5: exrand function

```

function y=exrand(lamada,n);
% this function is to return n expotional randoms
r=rand(1,n);
y=(-1/lamada)*log(r);

```

Appendix 6: mutisimulation function

```

function [a,Tas]=mutisimulation(Q,m);
% this function is to simulation the running of tollbooths. Q is traffic
% flow for per travel lane, n is the number of cars we are to simulate,m is
% the number of tollbooths for each travel lane. Here we are trying to get
% waiting time for the given condition (parameters).
%this function is to return the average waiting time Tas and max waiting
%time Tms
n=10000;
Tast=3.2; % average serve time for per car of the tollbooth
Tast1=2; % average serve time when they are to squeeze to one travel lane
lamada=3600/Q; % average arrival time interval.
Tstarray=Errand(1/Tast,n); % serve time for n cars
Titarray=exrand(1/lamada,n); % arrival time interval for n cars
% the first column is waiting time,second is arriving time, third is service
%time ,fourth is delaying time,fifth is departing time and leaving time
cars=zeros(n+1,5);
s=zeros(1,n);
t=0;
for i=1:n
    cars(i,2)=t;% arrival time
    cars(i,3)=Tstarray(i); % serve time
    t=t+Titarray(i);
end
t=zeros(1,m);
for i=1:n
    Tmin=min(t); % find the minimal time when car serves is done
    for j=1:m
        if t(j)==Tmin % which tollbooth to serve ith car .
            k=j;
            break;
        end
    end
end

```

```

% get cars' leaving tollbooths time
if cars(i,2)>=t(k)
    cars(i,5)=cars(i,2)+cars(i,3);
else
    cars(i,5)=t(k)+cars(i,3);
end
t(k)=cars(i,5);
s(i)=cars(i,5)-cars(i,2)-cars(i,3);
end
% bubble sort
temp=0;
for i=1:n-1
    for j=i+1:n
        if cars(i,5)>cars(j,5)
            cars(n+1,:)=cars(i,:);
            cars(i,:)=cars(j,:);
            cars(j,:)=cars(n+1,:);
        end
    end
end
t=0;
%serve time for cars squeezing to one travel lane
Tstarray=Errand(1/Tast1,n);
%cars' leaving time
for i=1:n
    if cars(i,5)>=t
        cars(i,5)=cars(i,5)+Tstarray(i);
    else
        cars(i,5)=t+Tstarray(i);
    end
    t=cars(i,5);
end
for i=1:n
    % serve time
    cars(i,3)=cars(i,3)+Tstarray(i);
    %delay time
    cars(i,4)=cars(i,5)-cars(i,2);
    %waiting time
    cars(i,1)=cars(i,4)-cars(i,3);
end
maxs=0;
sums=0;
for i=1:n
    % the sum of waiting time
    sums=sums+cars(i,1);
    if maxs<cars(i,1)
        maxs=cars(i,1); %max waiting time
    end
end
Tas=sums/n; %average waiting time
a=mean(s);% the first queueis average waiting time

```

```

if a<0.0001
    a=0;
end

```

Appendix 7: search function

```

%This function is to search the optimal number of tollbooths per travel lane
function optn=search
Q=1100;
m=10;
Tw1=zeros(1,m);
Tw2=zeros(1,m);
rand(1,sum(100*clock));
for n=1:m
    [Tw1(n),Tw2(n)]=mutisimulation(Q,n);
end
for n=2:m
    if (Tw2(n-1)-Tw2(n))/Tw2(n-1)<0.05
        optn=n-1;
        break;
    end
end
end

```

Appendix 8: probability function

```

function probability(Q,m);
% this function is to get the probability of interval of toll-departing
% time, which is shown in a figure
n=10000;
Tast=3.2; % average serve time for per car of the tollbooth
Tast1=2; % average serve time when they are to squeeze to one travel lane
lamada=3600/Q; % average arrival time interval.
Tstarray=Errand(1/Tast,n); % serve time for n cars
Titarray=exrand(1/lamada,n); % arrival time interval for n cars
% the first column of cars is arriving time,second is service time, third
% is departing time and leaving time
cars=zeros(n+1,3);
t=0;
for i=1:n
    cars(i,1)=t;% arrival time
    cars(i,2)=Tstarray(i); % serve time
    t=t+Titarray(i);
end
t=zeros(1,m);
for i=1:n
    Tmin=min(t); % find the minimal time when car serves is done
    for j=1:m
        if t(j)==Tmin % which tollbooth to serve car i.
            k=j;
            break;
        end
    end
end

```

```

end
% get cars' leaving tollbooths time
if cars(i,1)>=t(k)
    cars(i,3)=cars(i,1)+cars(i,2);
else
    cars(i,3)=t(k)+cars(i,2);
end
t(k)=cars(i,3);
end
% bubble sort
for i=1:n-1
    for j=i+1:n
        if cars(i,3)>cars(j,3)
            cars(n+1,:)=cars(i,:);
            cars(i,:)=cars(j,:);
            cars(j,:)=cars(n+1,:);
        end
    end
end
s=zeros(1,n);
%get departing time interval
for i=2:n
    s(i)=cars(i,3)-cars(i-1,3);
end
maxs=max(s);
count=zeros(1,101);
%get the probability of departing time interval
for i=1:n
    k=ceil(s(i)*100/maxs)+1;
    count(k)=count(k)+1;
end
for i=1:101
    count(i)=count(i)/(n-1);
end
for i=2:101
    count(i)=count(i)+count(i-1);
end
t=0:maxs/100:maxs;
plot(t,count,'r');

```

Appendix 9: mixsimulation function

```

function [a,Tas]=mixsimulation(Q,m);
% this function is to simulation the running of tollbooths with different
%service type. Q is traffic flow for per travel lane, n is the number of
%cars we are to simulate,m is
% the number of tollbooths for each travel lane. Here we are trying to get
% waiting time for the given condition (parameters).
%this function is to return the average waiting time Tas and max waiting
%time Tms
n=10000;

```

```

Tast=3.2; % average serve time for per E-ZPass car of the tollbooth
Tast1=2; % average serve time when they are to squeeze to one travel lane
Tast3=8; % average serve time for per cash pay car of the tollbooth
lamada=3600/Q; % average arrival time interval.
Tstarray=Errand(1/Tast,n); % serve time for n cars
Tstarray2=Errand(1/Tast3,n);
Titarray=exrand(1/lamada,n); % arrival time interval for n cars
% the first column is waiting time,second is arriving time, third is service
%time ,fourth is delaying time,fifth is departing time and leaving time
cars=zeros(n+1,6);
s=zeros(1,n);
t=0;
as1=zeros(1,m-1);
as2=zeros(1,m-1);
p=rand(1,n);
for i=1:n
    if p(i)<=0.7
        cars(i,6)=1;% service type
    else
        cars(i,6)=2;
    end
end
for i=1:n
    cars(i,2)=t;% arrival time
    t=t+Titarray(i);
    if cars(i,6)==1
        cars(i,3)=Tstarray(i); % serve time
    else
        cars(i,3)=Tstarray2(i);
    end
end
for h=1:(m-1)
    t1=zeros(1,h);
    t2=zeros(1,m-h);
    for i=1:n
        if cars(i,6)==1%E-ZPass
            Tmin=min(t1); % find the minimal time when car serves is done
            for j=1:h
                if t1(j)==Tmin % which tollbooth to serve ith car .
                    k=j;
                    break;
                end
            end
            % get cars' leaving tollbooths time
            if cars(i,2)>=t1(k)
                cars(i,5)=cars(i,2)+cars(i,3);
            else
                cars(i,5)=t1(k)+cars(i,3);
            end
            t1(k)=cars(i,5);
        else%cash pay

```

```

    Tmin=min(t2); % find the minimal time when car serves is done
    for j=1:m-h
        if t2(j)==Tmin % which tollbooth to serve ith car .
            k=j;
            break;
        end
    end
    % get cars' leaving tollbooths time
    if cars(i,2)>=t2(k)
        cars(i,5)=cars(i,2)+cars(i,3);
    else
        cars(i,5)=t2(k)+cars(i,3);
    end
    t2(k)=cars(i,5);
end
s(i)=cars(i,5)-cars(i,2)-cars(i,3);
end
% bubble sort
temp=0;
for i=1:n-1
    for j=i+1:n
        if cars(i,5)>cars(j,5)
            cars(n+1,:)=cars(i,:);
            cars(i,:)=cars(j,:);
            cars(j,:)=cars(n+1,:);
        end
    end
end
t=0;
%serve time for cars squeezing to one travel lane
Tstarray=Errand(1/Tast1,n);
%cars' leaving time
for i=1:n
    if cars(i,5)>=t
        cars(i,5)=cars(i,5)+Tstarray(i);
    else
        cars(i,5)=t+Tstarray(i);
    end
    t=cars(i,5);
end
for i=1:n
    % serve time
    cars(i,3)=cars(i,3)+Tstarray(i);
    %delay time
    cars(i,4)=cars(i,5)-cars(i,2);
    %waiting time
    cars(i,1)=cars(i,4)-cars(i,3);
end
maxs=0;
sums=0;
for i=1:n

```

```
% the sum of waiting time
sums=sums+cars(i,1);
if maxs<cars(i,1)
    maxs=cars(i,1); %max waiting time
end
end
as1(h)=sums/n; %average waiting time
as2(h)=mean(s);% the first queueis average waiting time
if as2(h)<0.0001
    as2(h)=0;
end
end
Tas=min(as1);
for i=1:m-1
    if as1(i)==Tas
        optn=i;
        a=as2(i);
        break;
    end
end
disp('Number of E-Z pass tollbooths');
disp(optn);
disp('Number of cash');
disp(m-optn);
```